

DD2459 Software Reliability

Lab - 2

Md Sakibul Islam

msis3@kth.se

Md Ahsanul Karim

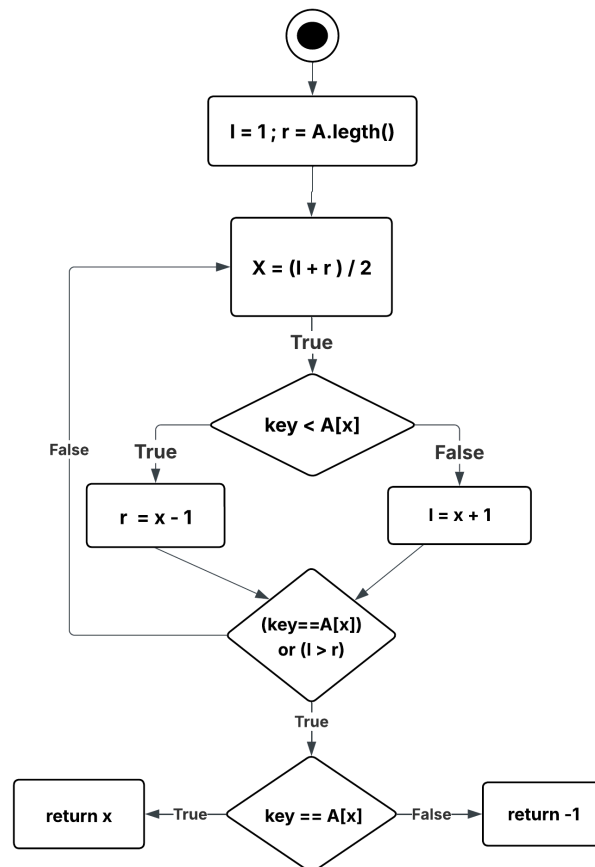
makari@kth.se

White-box Testing

Mar 3, 2026

Q1: Draw a condensation graph for the Binary Search algorithm (Algorithm 2)

Answer:



Q2 : Write JML pre/postconditions (**requires/ensures**) for Sorting, Searching, Membership, Binary Searching.

Answer: Sorting

```
/*@ public normal_behavior
   @ requires a != null;
   @ ensures \result != null && \result.length == a.length;
   @ ensures (\forall int i; 0 <= i && i < \result.length - 1; \result[i] <= \result[i + 1])
   */
public static int[] sort(int[] a) { 2 usages new }
```

Searching

```
/*@ public normal_behavior
   @ requires a != null;
   @ ensures (\result == -1) <==> (\forall int i; 0 <= i && i < a.length; a[i] != key);
   @ ensures (\result != -1) ==> (0 <= \result && \result < a.length && a[\result] == key);
   */
public static int search(int[] a, int key) { 3 usages new }
```

Membership

```
/*@ public normal_behavior
   @ requires a != null;
   @ ensures \result <==> (\exists int i; 0 <= i && i < a.length; a[i] == key);
   */
public static boolean membershipBySortThenBinarySearch(int[] a, int key) { 1 usage new }
```

Binary Searching

```
/*@ public normal_behavior
   @ requires a != null;
   @ requires (\forall int i; 0 <= i && i < a.length - 1; a[i] <= a[i + 1]);
   @ ensures (\result == -1) <==> (\forall int i; 0 <= i && i < a.length; a[i] != key);
   @ ensures (\result != -1) ==> (0 <= \result && \result < a.length && a[\result] == key);
   */
public static int binarySearch(int[] a, int key) { 1 usage new }
```

Q3: Implement 3 programs:

- **Program (i)** shows the implementation of bubble sort which is used to sort integer arrays of arbitrary length.

```
// (i) Sorting - Bubble Sort
public static int[] sort(int[] arr) { 1 usage  @ahsanbracu *
    int n = arr.length;
    int[] sorted = arr.clone();

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (sorted[j] > sorted[j + 1]) {
                int temp = sorted[j];
                sorted[j] = sorted[j + 1];
                sorted[j + 1] = temp;
            }
        }
    }
    return sorted;
}
```

- **Program(ii)** shows the implementation of membership queries on sorted array of arbitrary length using binary search.

```
// (ii) Binary Search
public static int search(int[] arr, int key) { 1 usage  @ahsanbracu *
    int l = 0, r = arr.length - 1;

    while (l <= r) {
        int mid = (l + r) / 2;

        if (arr[mid] == key)
            return mid;
        else if (key < arr[mid])
            r = mid - 1;
        else
            l = mid + 1;
    }

    return -1;
}
```

- **Program(ii)** Membership on an *unsorted* array by combining (i) + (ii)

```
// (iii) Membership – sort then search
public static int membership(int[] arr, int key) { 1 usage  ahsanbracu *
    int[] sorted = sort(arr);
    System.out.println(Arrays.toString(sorted));

    int result = search(sorted, key);
    return result != -1 ? 1 : 0;
}
```

Q4: Build a **random** and **pairwise** testing framework for program (iii):

To implement and test the six mutations, three programs were developed: bubble sort, binary search, and membership as described in Question 3. The correct implementation was used as the oracle to compare against each mutated version.

For random testing, a file of 100,000 test cases was generated with array size N and values drawn uniformly from the range 1-40. Each trial randomly samples from this file up to 2,000 times until a mismatch between the mutation output and the oracle output is found. If no mismatch is found within 2,000 attempts, the trial is capped and the next trial begins. The average number of attempts across all trials is reported.

For pairwise testing, a file of 174 test cases was generated using the PICT tool with 11 parameters, 10 array positions and 1 key each taking values from 1-10. The file is read sequentially and the row number of the first mismatch between the mutation and the oracle is reported as the minimum number of test cases needed to detect the bug.

For N=100 and N=500, the random test file was regenerated with the corresponding array size. For pairwise testing, the same 174-row file was used with the remaining positions padded with random values to reach the required array size.

The experiments were run for three array sizes: N=10, N=100, and N=500 to observe how detection efficiency changes with input size.

Table 1: Number of test cases needed to detect each mutation (N=10)

Mutation	Random (avg)	Pairwise (min)
Inner loop starts at j=1	1.082	1
"l < r" instead of "l <= r"	6.799	2
"key <= arr[x]" in final check	2.876	1
Loop condition "l > r", never executes	4.981	2
key "==" arr[x]" inverted loop condition	4.482	2
">=" instead of ">" in bubblesort	6.799	2

Table 2: Number of test cases needed to detect each mutation (N=100)

Mutation	Random (avg)	Pairwise (min)
Inner loop starts at j=1	1.063	1
"l < r" instead of "l <= r"	1.161	1
"key <= arr[x]" in final check	1.146	1
Loop condition "l > r", never executes	1.094	1
key "==" arr[x]" inverted loop condition	1.088	1
">=" instead of ">" in bubblesort	1.161	1

Table 3: Number of test cases needed to detect each mutation (N=500)

Mutation	Random (avg)	Pairwise (min)
Inner loop starts at j=1	1.025	1
"l < r" instead of "l <= r"	1.025	1
"key <= arr[x]" in final check	1.025	1

Loop condition " $l > r$ ", never executes	1.000	1
key " $== \text{arr}[x]$ " inverted loop condition	1.000	1
" $>=$ " instead of " $>$ " in bubblesort	1.025	1

Mutation 1

In the binary search, after the loop exits, the function should return the index x if the key is found, or -1 if not. A mutation is introduced by always returning x regardless of whether the key was found. Therefore, the binary search always returns the last computed midpoint index even when the key does not exist in the array.

```
int x = (l + r) / 2;
while (key != arr[x] && l <= r) {
    if (key < arr[x]) r = x - 1;
    else l = x + 1;
    x = (l + r) / 2;
}
return x; // BUG: should be (key == arr[x]) ? x : -1
```

Mutation 2

In the binary search loop, the search continues while the key is not found and the left index is less than or equal to the right index. A mutation is introduced by changing the condition to while the left index is strictly less than the right index. Therefore, when left equals right and the final single candidate the loop exits without checking it, causing the key to be missed.

```
int x = (l + r) / 2;
while (key != arr[x] && l < r) { // BUG: < instead of <=
    if (key < arr[x]) r = x - 1;
    else l = x + 1;
    x = (l + r) / 2;
}
return (key == arr[x]) ? x : -1;
```

Mutation 3

In the binary search, after the loop exits, the found index is returned if the key equals the element at position `x`, otherwise `-1` is returned. A mutation is introduced by changing this final comparison to return the index if the key is less than or equal to the element at position `x`.

Therefore, the function incorrectly returns an index even when the key is not present but a larger value exists at that position.

```
int x = (l + r) / 2;
while (key != arr[x] && l <= r) {
    if (key < arr[x]) r = x - 1;
    else l = x + 1;
    x = (l + r) / 2;
}
return (key <= arr[x]) ? x : -1; // BUG: <= instead of ==
```

Mutation 4

In the binary search loop, the search continues while the key is not found and the left index is less than or equal to the right index. A mutation is introduced by changing the loop condition to while the left index is greater than the right index. Since at the start left is always 0 and right is always `n-1`, this condition is never true and the loop never executes. Therefore, the binary search only returns the correct result when the key happens to be exactly at the initial midpoint of the array, and returns `-1` for all other cases.

```
private static int isMember(int[] arr, int key) { 1 usage 2 ahsanbracu
    int l = 0, r = arr.length - 1;
    int x = (l + r) / 2;
    while (key != arr[x] && l > r) { // BUG: l > r, always false
        if (key < arr[x]) r = x - 1;
        else l = x + 1;
        x = (l + r) / 2;
    }
    return (key == arr[x]) ? x : -1;
}
```

Mutation 5

In the binary search, after the loop exits, the correct index “x” is returned when the key is found. A mutation is introduced by returning “x-1” instead of “x”. Therefore, the function returns the wrong index where one position to the left of the actual found position.

```
private static int isMember(int[] arr, int key) { 1 usage 2 ahsanbracu
    int l = 0, r = arr.length - 1;
    int x = (l + r) / 2;
    while (key != arr[x] && l <= r) {
        if (key < arr[x]) r = x - 1;
        else l = x + 1;
        x = (l + r) / 2;
    }
    return (key == arr[x]) ? x - 1 : -1; // BUG: x-1 instead of x
}
```

Mutation 6

In the bubble sort, the inner loop comparison uses “>” to swap elements only when the left element is strictly greater than the right. A mutation is introduced by changing this to “>=”, causing equal elements to be swapped unnecessarily. Therefore, the sort becomes unstable and the array may not be correctly sorted in all cases, leading to incorrect binary search results.

```
int[] sorted = arr.clone();
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 2; j++) { // BUG: -2 instead of -1
        if (sorted[j] > sorted[j + 1]) {
            int temp = sorted[j];
            sorted[j] = sorted[j + 1];
            sorted[j + 1] = temp;
        }
    }
}
```

Variation in Testing Effort

When looking at the results for N=10, there is clear variation in testing effort between the injected errors. Mutations injected into the binary search (M1-M5) show varying detection rates depending on the nature of the bug. M1, which always returns the last

midpoint, is the easiest to detect with random testing (avg 1.08) because it fails on any test case where the key is not in the array, a very common condition. M2, which changes “`l <= r`” to “`l < r`”, is the hardest to detect with random testing (avg 6.79) because the bug only triggers when the key is the last single candidate when “`l == r`”, which is a rare condition in random testing. M3, M4 and M5 fall in between, requiring 2.87, 4.98 and 4.48 attempts on average respectively.

M6, which is injected into the bubblesort rather than the binary search, shows the same detection rate as M2 (avg 6.79) for random testing. This is because the “`>=`” bug in bubblesort only affects cases where equal elements exist in the array and their positions matter for the binary search result, which is relatively rare with random inputs.

Pairwise testing shows much less variation, all mutations are detected within 1-2 test cases regardless of the bug location or nature. This is because pairwise testing systematically covers all combinations of parameter values, ensuring boundary conditions like “`l == r`” and equal adjacent elements are always exercised. This demonstrates that pairwise testing is more robust than random testing for detecting bugs that only trigger under specific conditions.

Regarding injected locations, mutations in binary search (M1-M5) and bubblesort (M6) show similar detection difficulty for random testing, suggesting that the nature of the bug matters more than its location. However pairwise testing handles both locations equally efficiently, further confirming its systematic advantage over random testing.

Test Effort Comparison

When the table is investigated for array length 10, it can be observed that random testing requires more attempts on average to detect mutations compared to pairwise testing. Pairwise testing consistently detects all mutations within the first 2 test cases out of 174, while random testing requires between 1.08 and 6.79 attempts on average out of 100,000. Mutation 2 and Mutation 6 are the hardest to detect with random testing, requiring an average of 6.79 attempts, while pairwise testing detects both within 2 test cases. This demonstrates that pairwise testing is more systematic and efficient, as it is designed to cover all pairs of parameter values, ensuring that critical boundary conditions are always tested.

When the array length is increased to 100 and 500, the random testing average drops significantly to approximately 1.0 for all mutations. This is because with larger arrays and values in the range 1-40, the key is almost always present in the array due to the pigeonhole principle with 100 or 500 elements drawn from only 40 possible values, duplicates are guaranteed. This means the sort and search functions are almost always

exercising the found path, making most bugs detectable immediately. Pairwise testing also detects all mutations at row 1 for $N=100$ and $N=500$, confirming this observation.

Overall, pairwise testing demonstrates consistent and efficient bug detection across all array sizes, always finding mutations within the first 2 test cases. Random testing performs comparably for larger arrays due to the high probability of key presence, but is significantly less efficient for smaller arrays where the value range is sparse relative to the array size. Therefore, pairwise testing is the more reliable and efficient approach for mutation testing in this scenario.